

PROGRAMMATION FONCTIONNELLE 2 : PARTIEL 5

juin 2026 — Pierre Rousselin

Exercice 1 : Preuves de logique des propositions (12 points)

Dans chacun des cas suivants, donner :

- une preuve Rocq en programmation fonctionnelle, c'est-à-dire avec un seul terme de preuve dans exact ;
- une preuve Rocq avec des tactiques de plus haut niveau et exact seulement à la fin sur une hypothèse ;
- un arbre de dérivation en déduction naturelle.

Ici, A, B et C sont trois propositions (en Rocq, des types de sorte Prop) quelconques.

1. (4 points) Lemma ex5_1 : (A $\wedge$ B) $\wedge$ $\sim$ A $\rightarrow$ B.
 2. (4 points) Lemma ex5_2 : $\sim$ A $\wedge$ $\sim$ B $\rightarrow$ $\sim$ (A $\wedge$ B).
 3. (4 points) Lemma ex5_3 : (A $\rightarrow$ B $\wedge$ C) $\rightarrow$ $\sim$ C $\rightarrow$ A $\rightarrow$ B. Pour le terme de preuve, on pourra utiliser la fonction False_ind dont le type est : False_ind : forall (P : Prop), False $\rightarrow$ P.
-

Exercice 2 : Arbres d'expressions arithmétiques (15 points)

On considère le type inductif suivant, en Rocq, qui représente des expressions arithmétiques construites à partir de littéraux entiers, d'additions et de multiplications.

```
Inductive expr : Set :=  
  | Lit (n : nat)  
  | Add (e1 e2 : expr)  
  | Mul (e1 e2 : expr).
```

Par exemple, l'expression $(2 + 3) * 4$ se représente par :

```
Definition exple_7 := Mul (Add (Lit 2) (Lit 3)) (Lit 4).
```

1. (1 point) Représenter sous forme d'arbre (schéma) l'expression suivante, appelée exple_8 par la suite :

```
Definition exple_8 := Add (Lit 5) (Mul (Lit 2) (Lit 6)).
```

2. (2 points) On considère la fonction d'évaluation suivante :

```
Fixpoint eval (e : expr) : nat :=  
  match e with  
  | Lit n      => n  
  | Add e1 e2 => eval e1 + eval e2  
  | Mul e1 e2 => eval e1 * eval e2  
  end.
```

Évaluer pas à pas `eval exple_8` en étiquetant chaque étape de calcul par le commentaire correspondant dans la fonction.

3. (2 points) On considère la fonction suivante, qui compte le nombre de littéraux d'une expression :

```
Fixpoint size (e : expr) : nat :=
  match e with
  | Lit _      => 1                (* sL *)
  | Add e1 e2 => size e1 + size e2 (* sA *)
  | Mul e1 e2 => size e1 + size e2 (* sM *)
  end.
```

Évaluer pas à pas `size exple_8` en étiquetant chaque étape de calcul par le commentaire correspondant dans la fonction.

4. (4 points) On considère la fonction suivante :

```
Fixpoint doubleLits (e : expr) : expr :=
  match e with
  | Lit n      => Lit (2 * n)      (* dL *)
  | Add e1 e2 => Add (doubleLits e1) (doubleLits e2) (* dA *)
  | Mul e1 e2 => Mul (doubleLits e1) (doubleLits e2) (* dM *)
  end.
```

Donner la « preuve papier très détaillée » du lemme suivant :

```
Lemma size_doubleLits (e : expr) : size (doubleLits e) = size e.
```

5. (4 points) On considère la fonction suivante :

```
Fixpoint iter {B : Type}
  (fl : nat -> B) (fa fm : B -> B -> B) (e : expr) : B :=
  match e with
  | Lit n      => fl n
  | Add e1 e2 => fa (iter fl fa fm e1) (iter fl fa fm e2)
  | Mul e1 e2 => fm (iter fl fa fm e1) (iter fl fa fm e2)
  end.
```

Donner la « preuve papier très détaillée » du lemme suivant :

```
Lemma iter_eval (e : expr) :
  iter (fun n => n) (fun x y => x + y) (fun x y => x * y) e = eval e.
```

6. (2 points) En utilisant la fonction `iter`, sans récursivité (autre que celle déjà dans `iter`), définir `countMuls`, qui calcule le nombre de nœuds `Mul` d'une expression.

Exercice 3 : Prédicat inductif sur nat (13 points)

On se donne le prédicat inductif suivant (binaire sur nat) :

```
Inductive ecart : nat -> nat -> Prop :=
  | en  (n : nat) : ecart n n
  | eS1 (n m : nat) : ecart n m -> ecart n (S (S m))
  | eS2 (n m : nat) : ecart n m -> ecart (S (S n)) m.
```

- (1 point) Représenter les trois constructeurs sous la forme de règles d'inférence (utiliser le nom du constructeur comme nom de la règle).
- (4 points) Donner dans chaque cas suivant la preuve du fait demandé sous la forme d'un arbre de dérivation et en Rocq.

a) Lemma `ecart37` : `ecart 3 7`.

b) Lemma `ecart73` : `ecart 7 3`.

- (4 points) On considère le lemme et le début de preuve suivants :

```
Lemma ecart_diff (n m k : nat) : m = n + 2 * k -> ecart n m.
Proof.
  induction k as [| k IH] in m |- *.
```

Terminer la preuve en Rocq, en indiquant dans chaque cas les hypothèses et la formule à prouver.

- (4 points) On considère le lemme et le début de preuve suivants :

```
Lemma ecart_exists (n m : nat) : ecart n m ->
  (exists k, m = n + 2 * k) \/ (exists k, n = m + 2 * k).
Proof.
  intros H. induction H as [p | p q Hpq IHpq | p q Hpq IHpq].
```

Terminer la preuve, en donnant dans chaque cas les hypothèses et le but, puis la fin de la preuve en Rocq. Vous pouvez utiliser la tactique `lia` pour les égalités ou inégalités « évidentes » entre entiers naturels.

Exercice 4 : listes, prédicat universel, réflexion (20 points)

On considère le prédicat booléen suivant sur les listes, et le prédicat inductif (sur Prop) correspondant :

```
Fixpoint forallb {A : Type} (p : A -> bool) (l : list A) : bool :=
  match l with
  | []      => true
  | h :: t => p h && forallb p t
  end.
```

```
Inductive All {A : Type} (p : A -> Prop) : list A -> Prop :=
  | All_nil                               : All p []
  | All_cons (h : A) (t : list A) :
    p h -> All p t -> All p (h :: t).
```

On pose `p := fun n => 3 <= n` (le booléen « est supérieur ou égal à 3 »).

- (2 points) Évaluer pas à pas `forallb p [5; 10; 3; 7]`.
- (2 points) Prouver `All (fun n => 3 <= n) [5; 10; 3; 7]` (on utilisera les constructeurs `All_nil` et `All_cons`, et on pourra admettre les inégalités numériques évidentes).
- (5 points) Prouver le lemme suivant, en raisonnant par induction sur `l` :

```
Lemma forallb_All {A : Type} (p : A -> bool) (l : list A) :
  forallb p l = true -> All (fun x => p x = true) l.
```

4. (5 points) Prouver le lemme suivant, en raisonnant par induction sur la preuve de All :

```
Lemma All_forallb {A : Type} (p : A -> bool) (l : list A) :
  All (fun x => p x = true) l -> forallb p l = true.
```

5. (6 points) Prouver le lemme suivant (équivalence, à prouver dans les deux sens), en raisonnant par induction sur l1 :

```
Lemma All_app {A : Type} (p : A -> Prop) (l1 l2 : list A) :
  All p (l1 ++ l2) <-> All p l1 /\ All p l2.
```

Annexe 1 : règles de la déduction naturelle

$$\begin{array}{c}
\frac{}{\Gamma, A \vdash A} (Ax) \qquad \frac{\Gamma \vdash \text{False}}{\Gamma \vdash A} (\text{ex falso}) \\
\\
\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} (\wedge I) \quad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} (\wedge E-g) \quad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} (\wedge E-d) \\
\\
\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} (\vee I-g) \quad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} (\vee I-d) \quad \frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash P \quad \Gamma, B \vdash P}{\Gamma \vdash P} (\vee E) \\
\\
\frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} (\rightarrow I) \quad \frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} (\rightarrow E) \\
\\
\frac{\Gamma, x:T \vdash P}{\Gamma \vdash \forall x:T, P} (\forall I) \quad \frac{\Gamma \vdash \forall (x:T), P \quad \Gamma \vdash t:T}{\Gamma \vdash P\{x/t\}} (\forall E)
\end{array}$$

Quelques définitions standard en Rocq

```
Inductive bool := true | false.
Definition andb b1 b2 := if b1 then b2 else false.
Definition orb b1 b2 := if b1 then true else b2.
Notation "b1 && b2" := (andb b1 b2).
Notation "b1 || b2" := (orb b1 b2).

Inductive and (A B : Prop) : Prop := conj (hA : A) (hB : B).
Inductive or (A B : Prop) : Prop :=
  or_introl (hA : A) | or_intror (hB : B).

Inductive nat : Set := 0 : nat | S : nat -> nat.
(* Notations usuelles : 0 pour 0, 1 pour (S 0), etc *)
Fixpoint add (n m : nat) :=
  match n with
  | 0    => m
  | S p => S (add p m)
end.
```

```

Inductive list (A : Type) : Type :=
  nil : list A | cons : A -> list A -> list A.
(* Notation [] pour nil et h :: t pour (cons h t) *)

Inductive le (n : nat) : nat -> Prop :=
  | le_n : le n n
  | le_S (m : nat) : le n m -> le n (S m).
Notation "n <= m" := (le n m).

```

Liste de tactiques Rocq vues en cours, TD et TP

- **exact** : terminer la preuve en cours avec un terme de preuve
- **intros** : introduire des variables ou hypothèses depuis le but vers le contexte local
- **apply** : appliquer une implication (dans le but ou, avec `in`, dans une hypothèse) en remplissant les trous avec de l'unification
- **exfalso** : remplacer le but par `False`
- **destruct** : raisonner par cas sur une valeur en liant des variables éventuelles aux arguments éventuels du ou des constructeurs
- **left ou right** : choisir le premier ou le deuxième constructeur ; plus souvent utilisé pour prouver une disjonction
- **split** : utiliser l'unique constructeur ; plus souvent utilisé pour prouver une conjonction
- **reflexivity** : terminer la preuve en cours lorsque le but se réduit à `a = a`
- **rewrite** : remplacer un terme par un autre qui lui est égal
- **induction** : comme `destruct` mais avec des hypothèses d'induction dans le(s) cas d'argument(s) dont le type est le même que la valeur sur laquelle on raisonne par induction
- **specialize** : donner des arguments à une fonction ou, ce qui revient au même, instancier un lemme
- **simpl** : calculer en utilisant les définitions
- **discriminate** : prouver `False` à partir d'une égalité impossible entre constructeurs distincts
- **injection** : extraire les égalités des arguments à partir d'une égalité entre constructeurs identiques
- **lia** : résoudre automatiquement les buts arithmétiques linéaires sur `nat` ou `Z`